Rehtek Visitor Sign-In System: Rebuild Specification

Oct 29, 2026 · @Stephen Reh

A complete description of Rehtek Machine Co.'s visitor sign-in system, written so another developer or AI model can rebuild it from scratch. Secrets (PINs, spreadsheet and folder IDs, deployment links) are left out on purpose; where the system needs one, this document says where it's configured.

1. Purpose and requirements

What the system does

Rehtek is a CNC machine shop that makes parts controlled under U.S. export law (ITAR and EAR). Every visitor must be identified, checked against citizenship rules, approved when required, escorted, and logged. The system replaces a paper log and a Google Form with:

- **Self sign-in on the visitor's own phone,** from a QR code on a physical visitor badge or a QR code in the lobby. No app install and no account.
- **A front desk web app (the Visitor Log)** that admits visitors automatically when the rules allow, holds them when a person must decide, and keeps the audit record.
- **Badge tracking:** reusable physical badges with a QR code and an NFC tag. Scanning the QR shows who holds the badge; tapping the NFC tag on a desk reader signs the holder out.
- **A lobby sign-out kiosk:** a full-screen view of everyone inside, with a Clock out button per visitor and NFC tap-to-sign-out.

Compliance requirements it must meet

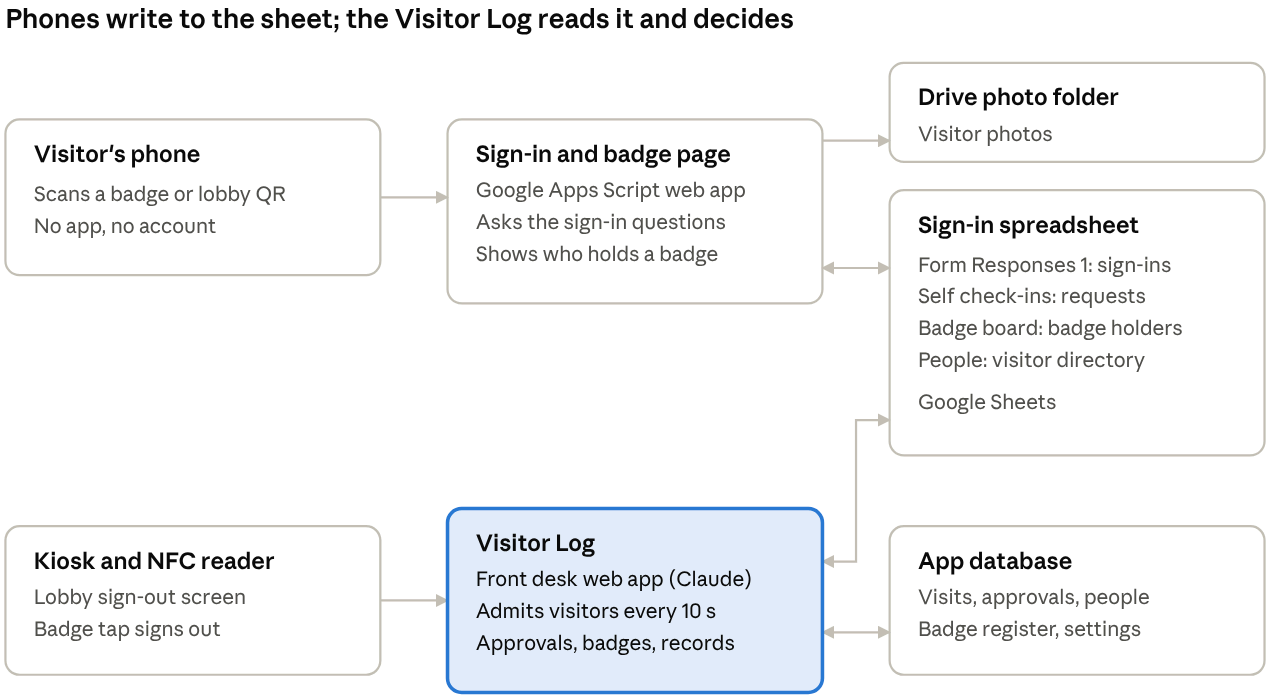
Requirement	Source	How the system meets it
Limit physical access to authorized individuals	CMMC PE.L1-3.10.1	Citizenship gate, Empowered Official approval for foreign persons, approved-area lists, door scans that block foreign nationals from controlled areas.
Monitor the facility	CMMC PE.L2-3.10.2	Live list of everyone inside, last area scanned, overdue alerts.
Escort visitors and monitor activity	CMMC PE.L1-3.10.3	Named escort at check-in, U.S.-person escort required for foreign nationals (Technology Control Plan), escort handover, host alert on arrival.
Keep audit logs of physical access	CMMC PE.L1-3.10.4	Every visit keeps a timestamped event history; monthly review; CSV export; 5-year retention.
Control and manage physical access devices	CMMC PE.L1-3.10.5	Badge register, badge type enforcement, return tracking at clock-out, lost-badge incidents, retirement.
Prevent unauthorized deemed exports	ITAR 22 CFR 120.50, EAR 15 CFR 734.13	Citizenship captured for every visitor, Consolidated Screening List check, Empowered Official decision with ITAR and EAR release fields, area restrictions.

Definitions used throughout

- **U.S. person:** a U.S. citizen, a lawful permanent resident (green card holder), or a protected individual (asylee or refugee). Everyone else is a **foreign person** (shown in the UI as "foreign national").
- **Empowered Official (EO):** the person who approves foreign-person visits and manages the screening list. In the app, EO and facility security lead are **Editors** (admin); front desk staff are **Contributors**.

- **Host:** the employee the visitor came to see. **Escort:** the employee who accompanies them.
- **Controlled area:** a shop area where export-controlled technical data or parts are visible.
- **Digital pass:** a temporary badge code ("D" plus five characters) used when the visitor has no physical badge yet.

2. System architecture



system architecture · 7 components

The visitor's phone only ever talks to the sign-in page. The sign-in page only ever writes to the spreadsheet and the photo folder. The Visitor Log, running on a front desk computer, polls the spreadsheet, makes every admission decision, keeps the official record in its own database, and writes results back to the spreadsheet for the phone to read.

The components

Component	Technology	Responsibility
Sign-in and badge page	Google Apps Script web app (one Code.gs file; server functions plus one HTML page held in a template string). Deployed as Execute as: owner, Who has access: Anyone.	The only public surface. Free badge: sign-in or check-in. Badge in use: photo, name and badge number, with "See more" behind a staff PIN. No badge in the link: lobby sign-in. Writes requests and sign-ins; never decides admission.
Sign-in spreadsheet	Google Sheets, owned by the shop's Google account. Four tabs (section 4).	The message bus between phones and the Visitor Log, and the permanent store of sign-in answers.
Drive photo folder	Google Drive folder.	Holds visitor photos uploaded from the sign-in page. Linked from the sign-in row.
Visitor Log	A single-file HTML web app hosted as a Claude artifact. Uses a document database, a file store, a download capability, the signed-in user's identity, and the Google Sheets and Google Drive connectors.	Front desk app: admission rules, badge assignment, approvals, screening, clock in and out, records, badge register, kiosk, NFC. Writes the Badge board, People and request results back to the sheet.
App database	Key-value document database belonging to the Visitor Log (collections of JSON documents with live subscriptions and per-path access rules).	The system of record: visits, approvals, directory, badge register, settings, monthly reviews.
Kiosk and NFC reader	A lobby screen running the Visitor Log in kiosk mode; a USB NFC reader that acts as a keyboard (types the tag number, then Enter).	Visitor self sign-out by badge tap or button.

Component	Technology	Responsibility
Physical badges	Reusable cards, numbered V1 to V25, each with a QR code (link to the sign-in page with ?badge=Vn) and an NFC sticker.	Identify which visitor holds which badge.

Why it's split this way

- **Visitors have no accounts.** Only a public web app (Apps Script with access Anyone) can serve them without a login. The Visitor Log requires the staff member's Claude sign-in, so it can't be the public page.
- **The public page can't decide anything.** It runs with the owner's Google permissions but holds no rules. All admission logic lives in the Visitor Log, which keeps the audit trail. If the front desk computer is off, sign-ins wait in the sheet and are processed when it comes back.
- **The spreadsheet is the only shared channel** both sides can reach: the Apps Script reads and writes it natively, the Visitor Log through its Google Sheets connector.

Request lifecycle in one paragraph

A phone action appends a row (a sign-in row on Form Responses 1, a request row on Self check-ins, or both). Every 10 seconds the Visitor Log reads new rows, applies the admission rules, creates or updates the visit in its database, assigns a badge, then writes a Result value onto the request row ("in V3", "waiting: …", "out", "error: …") and rewrites the Badge board and People tabs. The phone polls its request's Result every 3 seconds (every 10 while waiting) and shows the outcome.

3. Data model (the Visitor Log database)

A document database: collections of JSON documents addressed as `collection/docId`, with live snapshot subscriptions. Timestamps are ISO 8601 strings in UTC (`2026-10-29T14:05:09.000Z`); dates are local `YYYY-MM-DD` strings. User references are `{ id, name }` objects (or `{ name }` for someone typed in by hand).

Access rules

Path	Read	Write	Why
everything (default)	Contributor and up	Contributor and up	Front desk works here.
`approvals/`	Contributor and up	Editor (admin) only	Only the Empowered Official decides.
`reviews/`	Contributor and up	Editor only	Monthly sign-off.
`config/`	Contributor and up	Editor only	Badge register, settings, screening list pointer.
`badgeboard/`	**Viewer** and up	Contributor and up	Viewer-level staff can look up a badge holder without seeing the log.
`staffpin/`	Viewer and up	Editor only	Hashed staff PIN for badge lookups.

`visits/{visitId}`: one visit (the core record)

Field	Type	Meaning
`name`, `company`	string	Visitor's full name and company.
`date`, `endDate`	YYYY-MM-DD	Visit day; optional last day of a multi-day visit.
`arriveTime`, `leaveTime`	HH:MM	Optional expected window (drives the "Arriving now" strip from 30 min before arrival).
`purpose`	string	Why they're visiting.
`areas`	string[]	Area ids they may enter (for example `lobby`, `office`, `shop`, `inspection`); some areas are flagged controlled.
`citizenship`	enum	`us_citizen`, `permanent_resident`, `protected`, `foreign`. The first three are U.S. persons.

Field	Type	Meaning
`country`	string	Country of citizenship.
`host`, `escort`	user ref	Who they're seeing; who escorts them.
`walkin`	boolean	True when created from a sign-in rather than pre-registered.
`source`	enum	`form` (sign-in page or old form), `self` (badge-page check-in), `reentry` (front desk "Clock back in"), or absent for pre-registration.
`photoId`	string	Drive file id of the visitor photo.
`personId`	string	Link to `people/{id}`.
`wantBadge`	string	A badge number the visitor took while still waiting for approval; given to them when admitted.
`checkin`	object or absent	Present once admitted (below).
`checkout`	object or absent	`{ at, by, badgeReturned, stayedInAreas, note }`.
`events`	object[]	Append-only history: `{ at, by, type, note }`. Types: `registered`, `checkin`, `checkout`, `note`, `incident`, `correction`, `escort`, `area`, `approval`.

`checkin`: `{ at, by, source, signedInAt, idType, idChecked, citizenshipConfirmed, ackName, ackVersion, ackAt, ackSource, badge, badgeKind ("physical" or "digital"), badgeType ("visitor", "foreign_national" or "any"), noPhysical, escort, tcpEscort, screening: { result, by, at, hits }, photoId, badgeTakenAt }`.

`approvals/{visitId}`: the Empowered Official's decision

`{ decision: "approved" | "limited" | "denied", itar: release decision, ear: release decision, license: string, screening: "no_match" | "possible", screenedOn: date, note, at, by }`. "Limited" means approved for non-controlled areas

only. Live list loads the last 180 days; older approvals are fetched on demand when an old month is opened.

`signins/{key}` : processing state of each sign-in row

Key: `s` plus a hash of the row (form rows) or `q` plus a hash of the request id (badge-page requests). `{ status: "waiting" | "used" | "dismissed", visitId, reason, at, person: { status: "new" | "linked" | "confirm" | "pending", pid, changed[], candidates[] } }`. Waiting reasons: `citizenship`, `citizenship_changed`, `approval`, `denied`, `screening`, `badge_in_use`, `badge_type`, `badge_invalid`. Only the last 3 days are loaded.

`people/{personId}` : visitor directory

Id: `p` plus a hash of the email, or of normalized name plus company. `{ name, first, last, company, companyKey, nameKey, keys[] (every name|company key ever used), email, citizenship, citText, country, photoId, photoLink, firstSeen, lastSeen, visits, visitIds[] (last 50), lastHost, lastPurpose, changes[] (last 50: { at, field, from, to, source, visitId }), createdAt, updatedAt }`. A one-time backfill builds it from past visits and sign-ins; `meta/people` records that it ran.

`badgeboard/{badgeNo}` : live badge holder, readable by every staff level

`{ badge, type, retired, status: "in use" | "available" | "not returned" | "retired", holder: { name, company, kind, escort, host, areas[], since, idChecked, visitId, photo (small JPEG data URL), photoId } | null, updatedAt }`.

`config/` documents (Editor-only)

- `config/badges` : `{ list: [{ no, type, note, retired, nfc (tag number), addedAt, addedBy, nfcAt }], removed: [{ no, type, at, by, reason }] }`. Badge numbers are 1 to 20 of letters, digits, dash and dot.
- `config/form` : `{ sheetUrl, sheetId }`, the sign-in spreadsheet.
- `config/publicBadges` : `{ url }`, the deployed sign-in page link.
- `config/screening` : `{ assetId, loadedAt, rows }`, pointer to the uploaded Consolidated Screening List.

Other collections

- `staffpin/main`: `{ salt, hash }` where hash = SHA-256 of `salt:PIN`, hex.
- `reviews/{YYYY-MM}`: monthly review sign-off with counts.
- `locks/signins`: `{ device, until, by }`, a 75-second lease so only one front desk screen admits sign-ins at a time.
- `meta/people`: directory backfill marker.

4. Spreadsheet tabs

One Google spreadsheet, four tabs. Row 1 of every tab is a header row; both sides find columns **by header name** (case-insensitive substring), never by position, so columns can be reordered. Timestamps are written as `M/D/YYYY H:MM:SS` in the shop's time zone, which is what Google Forms used, so old and new rows parse the same way.

Form Responses 1: every sign-in (written by the sign-in page)

The name is historical: it began as a Google Form response sheet. The Google Form is retired; the sign-in page now appends rows in the same layout. The Visitor Log finds the tab by that name, or by any tab whose header contains both "first name" and "company".

Column header	Content
Timestamp	When they signed in.
Email Address	Blank (kept for old rows).
First Name, Last Name	As on their photo ID.
Company	Or "N/A" for individuals.
Who are you visiting at Rehtek?	Free text; matched to a staff member when possible.
Purpose of your visit	Free text.
Citizenship status	One of exactly four strings: `U.S. citizen`, `Lawful permanent resident (green card holder)`, `Protected individual (asylee or refugee)`, `Not a U.S. citizen or permanent resident`.
Country of citizenship	"United States" for U.S. citizens; typed otherwise.

Column header	Content
Visitor acknowledgment	Non-empty text means they agreed (the electronic signature), for example "I agree to the visitor rules (signed on the Rehtek sign-in page)". The front desk's "Clock back in" writes "Re-confirmed at the front desk on <date>", which marks the row as a re-entry.
Upload a picture of yourself	`https://drive.google.com/open?id=<fileId>`.

Self check-ins: requests from the badge page (written by both sides)

Column	Written by	Content
Timestamp	page	When the request was made.
Request	page	A unique id (UUID); the phone polls by it.
Action	page	`check in`, `take badge`, `hold for form`, or `log out` (section 6).
Badge	page	The badge number in the visitor's hand.
Person ID	page	Directory id when known; blank when the person was found only in sign-in history.
Email, Name, Company, Citizenship status, Country, Visiting, Purpose	page	Today's answers (blank where the action doesn't need them).
Rules	page	"Agreed to the visitor rules on the badge page" for check-ins.
Photo ID	page	Drive file id, when known.

Column	Written by	Content
Result	**Visitor Log**	Empty until processed. Then `in V3`, `waiting: <reason in plain words>`, `out`, `error: <message>`, or `expired`. Rows that are empty or start with "waiting" are re-evaluated on every poll.

Badge board: who holds each badge (written by the Visitor Log)

Columns: Badge, Type, Status (`in use`, `available`, `not returned`, `retired`), Name, Company, Kind (`Visitor` or `Foreign national`), Escort, Visiting, Areas (semicolon list), In since, ID checked (`yes` or `no`), Photo ID, Updated. The Visitor Log rewrites the whole block whenever its content changes, padding with blank rows to clear badges that were deleted.

People: the visitor directory, for lookups (written by the Visitor Log)

Columns: ID, Name, Email, Company, Citizenship status, Country, Photo ID, Last seen (date), Name key (normalized, section 6), Inside (blank, a physical badge number, or `no badge` when they're inside on a digital pass). Rewritten when the directory or who's inside changes.

Who writes what

- **The sign-in page never writes** the Badge board, People or Result columns. It only appends rows.
- **The Visitor Log never appends** sign-in rows except for the front desk "Clock back in" re-entry, and never deletes rows.
- `setup()` in the Apps Script creates the Badge board, People and Self check-ins tabs with their headers if they're missing; it never clears data.

5. The public sign-in and badge page

One Google Apps Script project, container-bound to the sign-in spreadsheet (Extensions, Apps Script), deployed as a web app. `doGet(e)` returns one HTML page (held in a template-literal constant `PAGE`, with the badge number injected as JSON). The page calls

server functions with `google.script.run`. It works on any phone browser; no sign-in.

Two modes, chosen by the link

- **Badge mode:** `<web app URL>?badge=V12`, printed as the QR code on badge V12. The badge parameter is cleaned to letters, digits, `_ . ~ -` (max 20).
- **Lobby mode:** the same URL with no badge, printed on the lobby poster. The visitor signs in; the front desk hands them a badge.

Screens, in order

1. **Status check (badge mode).** `status(badge)` reads the Badge board row: `unknown` (not a Rehtek badge, take it to the front desk), `retired`, `not returned`, `in use`, or `available`. A missing Badge board tab shows "not set up yet".
2. **Name and company.** Two fields, both required, then **Check in with this badge** (lobby: **Sign in**). `findPeople(name, company)`.
3. **Found once:** if they're already inside today, go to step 4; otherwise go to step 5. **Found several:** "Which one is you?" list showing name, company and last visit. **Not found:** step 6.
4. **Already inside (take badge).** "Welcome back, Dana. You're already signed in today" (or "...with badge V3") and one button, **Use badge V12**: `takeBadge(token, badge)`. Lobby mode just says they're already signed in.
5. **Quick confirmation (returning visitor).** Pre-filled company, citizenship status (select of the four strings) and country, all editable; then **Who are you here to see?** and **Why are you here?** (required), the five visitor rules, and the agreement checkbox (required). **Check in**: `checkIn(token, badge, answers)`.
6. **In-app sign-in (new visitor).** Shows the intro text, then the full question set below, with first and last name pre-filled from step 2. **Sign in with badge V12** (lobby: **Sign in**): `signUp(badge, answers)`.
7. **Waiting for the result** (badge mode). Polls `requestResult(request)` every 3 seconds (every 10 while the result is "waiting"). `in V3`: "You're checked in. Wear badge V3 where it can be seen. Your escort will meet you at the front desk." `waiting: ...`: "Please wait at the front desk" plus the reason; keeps polling, so it flips to checked in the moment the Empowered Official approves. `error: ...`: the message. After about 2 minutes with no result: "Your check-in is saved. The front desk will finish it shortly."
8. **Signed in** (lobby mode): "You're signed in. Please show your government photo ID at the front desk."

Badge in use: what anyone who scans sees

Only the holder's **photo, name, and "Wearing badge V12"** (`holder(badge)`). A small **See more** button asks for the staff PIN; `lookup(badge, pin)` then returns company, kind (visitor or foreign national, with the escort warning), escort, who they're visiting, approved areas, time in, and whether their ID was checked.

The in-app sign-in questions (the retired Google Form's, word for word)

Intro, four paragraphs:

> Welcome to Rehtek Machine Co. Please sign in before your visit. It takes about two minutes.
>
> Some of the parts we make are controlled under U.S. export laws (ITAR and EAR). To follow these laws, we need to know each visitor's citizenship status before they enter our shop.
>
> We use this information only to manage your visit and meet U.S. security and export requirements. Please don't enter ID numbers or other sensitive details.
>
> Need help or don't have a phone? Ask at the front office.

Question	Required	Hint
First name	yes	As shown on your photo ID.
Last name	yes	As shown on your photo ID.
A picture of yourself	yes	File input with `accept="image/*" capture="user"` (opens the front camera). Resized on the phone to at most 900 px on the long side, JPEG quality 0.85, about 100 KB; preview shown.
Company	yes	Write “N/A” if you're visiting on your own.
Who are you visiting at Rehtek?	yes	The Rehtek employee you're meeting.

Question	Required	Hint
Purpose of your visit	yes	For example: quality audit, first article review, equipment service.
Citizenship status	yes	Select of the four exact strings (section 4).
Country of citizenship	only when not a U.S. citizen	Shown only for the other three choices.
Visitor acknowledgment	yes	Checkbox: "I agree to the visitor rules. Checking this box is my electronic signature on these rules."

The visitor rules, shown above the checkbox:

1. I will stay with my Rehtek escort at all times while I am past the front desk.
2. I will not take photos, video or audio recordings, and I will keep my phone with the front desk or switched off in controlled areas.
3. Drawings, programs, parts and processes I see may be controlled under the International Traffic in Arms Regulations (ITAR) and are confidential to Rehtek and its customers. I will not copy, remove or disclose them.
4. The citizenship status I gave Rehtek is true. I will tell my escort if it changes.
5. I will return my visitor badge before I leave.

Server functions

Function	Does	Returns
`setup()`	Creates the Badge board, People and Self check-ins tabs with headers if missing; triggers the Drive permission prompt. Run once by hand.	nothing
`doGet(e)`	Serves the page with the cleaned badge number.	HTML
`status(badge)`	Badge board state, never a name.	`{ badge, state }`

Function	Does	Returns
`holder(badge)`	Photo (Drive thumbnail as a data URL, cached 6 h), name, badge, for a badge in use.	`{ state, name, photo, badge }`
`findPeople(name, company)`	Matches the People tab, then falls back to sign-in history in Form Responses 1 (newest first, one per name and company). Issues a 15-minute token per match so later calls never take raw ids from the phone.	`{ matches: [{ token, name, company, last, inside, citizenship, country }] }` or `{ error: "fullname"
`checkIn(token, badge, answers)`	Badge mode: refuses a badge in use or out of service; appends a `check in` request (deduplicated within 10 minutes). Lobby mode: appends a sign-in row built from the person on file plus today's answers.	`{ request }` or `{ signedIn: true }`
`takeBadge(token, badge)`	Appends a `take badge` request.	`{ request }`
`signUp(badge, answers)`	Validates every answer and the photo (must be a JPEG, PNG or WebP data URL under about 3 MB), saves the photo to the photo folder, appends the sign-in row; in badge mode also appends a `hold for form` request for that badge.	`{ request }`, `{ signedIn: true }`, or `{ error: "missing"
`requestResult(request)`	Reads that request's Result cell.	`{ result }`
`lookup(badge, pin)`	Staff view after the PIN check.	full holder details or `{ error: "pin"

All writes take a script lock (`LockService`, 10 s) so two phones can't interleave rows. The photo folder is the script property `PHOTO_FOLDER_ID`, else a built-in default (the old form's upload folder), else a folder named "Rehtek visitor photos" is found or created.

6. Visitor Log processing

Everything below runs inside the Visitor Log on whichever front desk screen has it open. If no screen is open, rows wait in the sheet and are processed on the next poll.

The 10-second cycle

Every 10 seconds (while the tab is visible), `fetchSignins` runs these steps in order:

1. **Read new sign-in rows.** First read of a session: the whole Form Responses tab (`A:Z`). After that, only from 5 rows above the last known end, merged onto the cached copy, so each poll is small. A full re-read happens every 10 minutes, or immediately if the tab shrank (rows deleted).
2. **Load holds.** Read the Self check-ins tab once; keep the pending `hold for form` rows (no Result, today, under 45 minutes old). The same read is reused by step 4.
3. **Process sign-ins** (`processSignins`): today's rows whose state is new or waiting.
4. **Process requests** (`processSelf`): Self check-ins rows with an empty Result or one starting with "waiting".

Only one screen processes at a time. Before steps 3 and 4 it takes a 75-second lease in `locks/signins` (read, write its device id, read back to confirm); another screen skips processing while a fresh lease belongs to someone else. A screen that's closed loses its lease within 75 seconds.

Parsing a sign-in row

Columns are found by header substring: "timestamp", "first name", "last name", "company", "visiting", "purpose", "citizenship status", "country", "acknowledg" or "agree", "picture" or "photo", "email". The citizenship text maps to the enum: contains "not a u.s." gives `foreign`; "permanent" gives `permanent_resident`; "protected" gives `protected`; "citizen" gives `us_citizen`. The photo id is the 25+ character id inside the Drive link. The row key is `s` plus a hash of timestamp and name. A row is a **re-entry** when its acknowledgment text starts with "Re-confirmed".

Admission rules (`admitSignin`), in this exact order

1. **Find or create the visit.** Match a pre-registered visit for today that isn't checked in: name tokens must overlap (fuzzy, below) on at least 2 tokens, company words add score. None: create a walk-in visit (`walkin: true`, areas default to lobby and offices).
2. **Find the person** in the directory (below). A badge-page request carrying a Person ID uses it directly.
3. **Hold checks.** The first that applies puts the sign-in in `waiting` with that reason and stops:
   - `citizenship`: a pre-registered visit whose citizenship differs from what they entered.
   - `citizenship_changed`: a returning visitor whose citizenship differs from their directory record.
   - `approval` / `denied`: a foreign person with no Empowered Official approval for this visit, or a denial.
   - `screening`: a possible match on the Consolidated Screening List (section 9) not yet cleared.
   - `badge_invalid`, `badge_in_use`, `badge_type`: only for a badge-page request whose scanned badge is unregistered or retired, held by someone else, or the wrong type for the visitor.
4. **Choose the badge:**
   - The badge the visitor scanned (a badge-page request), if it passed step 3.
   - Otherwise a held badge: one the visitor took while waiting (`visit.wantBadge`), or a pending `hold for form` row for the same name and company. It must still be registered, not retired, free, and the right type; if not, it's ignored.
   - Otherwise, a front desk "Clock back in" re-entry gets the lowest-numbered free badge of the right type.
   - Otherwise a **digital pass**: `D` plus 5 random characters from `crypto.getRandomValues`. Form sign-ins start on a digital pass on purpose, because visitors take their own badge from the rack.
   - A badge just issued is locked for 120 seconds in memory so two admissions can't pick it at once.
5. **Admit:** write `checkin` (source, badge, kind, type, escort, screening result) and a `checkin` event: "Signed in on the visitor form at 2:31 PM; let in automatically with digital pass DK7Q2M until they take a badge."

6. **Link the person** (below) and mark the sign-in `used`.

The Empowered Official can force a waiting `citizenship`, `citizenship_changed` or `screening` hold through with **Cleared, let in**; approvals go through the approval dialog; badge holds go to the front desk.

Badge-page request actions (`processSelf`)

Action	What the Visitor Log does	Result written
`check in`	If the person is already inside today: give them the scanned badge (replacing a digital pass or another badge). If they have a waiting visit today: hold the badge on that visit. Otherwise run the admission rules with the scanned badge. Dismissed by the desk: error. Rules not agreed: waiting.	`in V12`, `waiting: ...`, `error: ...`
`take badge`	Person already signed in today: give them the badge. Waiting: hold it. Neither: error.	`in V12`, `waiting: ...`, `error: We couldn't find your sign-in for today.`
`hold for form`	Written by `signUp` in badge mode. Once their sign-in row is admitted, give them the badge; while waiting for approval, hold it; until their visit exists, leave Result empty; after 45 minutes, `expired`.	`in V12`, `waiting: ...`, empty, `expired`
`log out`	Find who holds the badge; the name must match; write `checkout` with badge returned.	`out`, `error: That badge belongs to someone else.`

Any row not from today gets `expired`. **Giving a badge** to someone already inside: the badge must be registered, not retired, not held by anyone else, and the right type (`any` fits everyone; `visitor` U.S. persons; `foreign_national` foreign persons); then `checkin.badge`, `badgeKind: "physical"`, `badgeType` and `badgeTakenAt` are updated, with a note event ("Took badge V12 on its badge page after signing in on the visitor form; replaces digital pass DK7Q2M").

Recognizing people (the directory)

Normalization. `nameKey`: remove accents (NFD, strip combining marks), uppercase, replace anything not A to Z or 0 to 9 with a space, collapse spaces, and drop one-letter tokens. Company words: the same, plus drop INC, LLC, LTD, CO, CORP, CORPORATION, COMPANY, THE, GROUP, LP, PLC, GMBH, AND, OF. **`near(a, b)`**: equal, or both at least 5 letters long and one insertion, deletion or substitution apart. **Same company:** both blank, or any company word of 3 or more letters is `near` a word of the other.

Matching rules, first that applies:

1. Same email: same person.
2. Same name key and same company: same person (only one may match).
3. Same name, different company (and no conflicting email): **ask**. The sign-in is still admitted, but the person link waits; a "Same person?" card on the Visitors tab offers **Same as <old company>, update** or **New person**.
4. Otherwise a new person.

Updating a returning visitor: company, citizenship, country, email and photo are overwritten with today's non-empty values; each change is logged in `changes[]` (at, field, from, to, source, visitId); the visit id is added and the visit count raised once per visit. Citizenship never changes silently: a difference holds the sign-in first (rule `citizenship_changed`), and the profile changes only after the Empowered Official clears it.

Badge-page lookups use the People tab: the first and last name tokens must each be `near`, and a company word must be `near`. When nothing matches there, the page searches sign-in history the same way.

Writing back to the sheet

After any change (debounced 1.5 s), the Visitor Log rebuilds the badge board in its database (adding a 220 px photo thumbnail when the photo has loaded), then rewrites the Badge board and People tabs **only when their content changed** (compared as JSON). A failed write waits 2 minutes before retrying; a missing tab shows "The Badge board tab doesn't exist yet. Run setup in the Apps Script." in Settings. Visitor text written to the sheet has leading `= + - @` stripped so it can never become a formula.

Each screen also reloads itself nightly around 3 AM (waiting if a dialog is open), so screens left open for days, like the kiosk, pick up new versions.

7. Visitor Log screens

One self-contained HTML file (CSS and JavaScript inline; QR libraries loaded from a CDN). Layout: a 232 px navy sidebar on desktop that becomes a bottom tab bar on phones; a top bar with the page title, a live-sync pill, **Sign-out kiosk**, the scan button and **New visit**. Light and dark themes follow the device setting.

Roles come from the signed-in user's permission on the app: Editors (Empowered Official, facility security lead) can approve, change settings and manage badges; Contributors (front desk) run everything else; Viewers see only the badge lookup screen.

Inside now (home)

- **Badge cards** for everyone checked in and not out, styled like real badges: a coloured top band (green for U.S. visitors, amber for foreign nationals, grey for digital passes), photo or coloured initials, name, company, badge number, escort, time in, and a red **Clock out** button.
- **Chips:** count inside, free visitor badges, free foreign-national badges.
- **Arriving now:** pre-registered visitors whose window starts within 30 minutes, each with **Clock in**.
- **Needs attention:** waiting sign-ins, each with its reason in plain words; the Empowered Official sees **Cleared, let in** where allowed; everyone sees **Dismiss**. A banner points to the Visitors tab when "Same person?" decisions are pending.
- **Host alerts:** when someone checks in for you (the host) and you didn't do it yourself, a banner says they've arrived; the browser tab title shows "(1) Visitor arrived".
- Tapping a card opens a **side panel**: full details, escort, ID check, areas, screening result, the event history, and actions (change escort, mark ID checked, report an incident, correct a field).

Clocking out from the app

Clock out on a physical badge asks "Is badge V3 back?" in the card: **Yes** writes checkout with the badge returned; **Badge missing** writes checkout with the badge not returned and an incident event, and the badge shows as not returned until found or retired. A digital pass clocks out straight away. Every clock-out shows a toast with **Undo** for 8 seconds.

Clock in dialog (for pre-registered visitors or anyone the desk checks in)

Badge picker (free badges of the right type first, or digital pass), escort picker (staff list or typed name), screening result (automatic search shown, or a manual confirmation), and three required checkboxes: photo ID checked, citizenship confirmed against the ID, visitor rules agreed. Foreign nationals also require **U.S.-person escort at all times (TCP)**. A scanned badge is validated against the register (exists, not retired, right type, not in use).

New visit (pre-registration)

Name, company, date and optional last day, optional arrive and leave times (leave must be after arrive), purpose, host, escort, areas (checkboxes; controlled areas marked), citizenship and country. Foreign persons go to the Empowered Official's approval queue, screened against the list.

Visitors

The directory: search by name, company, host or email; each row shows photo, name, company, last visit, visit count and a recent change ("Updated company and email on Oct 23"). **Details** shows the profile, every information change with source and date, and recent visits. **Clock back in** asks for today's host and purpose, reconfirms citizenship and the rules, and appends a re-entry row to the sign-in sheet so it goes through the normal admission rules. "Same person?" cards sit at the top when a match needs a human.

Expected

Today's pre-registered visits with **Clock in**, the next 14 days, and visitors who already left today.

Approvals (Empowered Official)

The queue of foreign-person visits. The approval dialog shows the screening search, then: decision (approved, approved for non-controlled areas only, denied), ITAR release, EAR release, license number, note. An ITAR or EAR answer of "unauthorized" blocks "approved as requested". Decided visits are listed below.

Records

A date-range log and a month view. Monthly stats map to the CMMC practices (visits, foreign-person visits with and without approval, escorts named, badges not returned, overdue visits) and **Mark month reviewed** (Editors). **Export CSV** includes areas entered,

escort changes, check-in screening and release decisions; cells that start with `= + - @` are prefixed with an apostrophe to stop spreadsheet formulas.

Settings (Editors)

- **Visitor sign-in:** the sign-in spreadsheet link; **Show the lobby QR code** (opens the sign-in page with no badge).
- **Public badge links:** the deployed sign-in page URL (must match `https://script.google.com/macros/s/.../exec`), setup steps, and the status of the Badge board and People writes.
- **Badge lookup PIN:** sets the hashed staff PIN for in-app badge lookups (4 to 8 digits).
- **Screening list:** upload the trade.gov Consolidated Screening List CSV; shows its age and warns after 7 days.
- **QR codes to print:** lobby poster, area door signs (`RVL:A:<area>`), badge labels.

Scan

Camera scanning with the browser's BarcodeDetector or jsQR 1.4.0, plus photo upload and typed fallback. Understands badge links (`...#badge-V12` or `...?badge=V12`), the older `RVL:B:<badge>` and door codes `RVL:A:<area>`. Scanning a door sets the area; the next badge scan logs that the holder entered it, and blocks foreign nationals approved for non-controlled areas from controlled ones. Scanning a badge also offers **Take over as escort**.

Badge lookup inside the app

Opening the app with `#badge-V12` shows a full-screen badge card fed from `badgeboard/V12` (live): photo, name, company, kind, escort, host, areas, time in, ID check. It asks for the staff PIN first when one is set (unlocked 15 minutes per device, 5 wrong tries pauses a minute). Viewer-level users see only this screen.

8. Badges, NFC sign-out, kiosk and QR codes

Badge register (Badges page)

- **Types:** `any` (one design for everyone, the default for Rehtek's blue-lanyard badges), `visitor` (U.S. persons only), `foreign_national`.

- **Add** a numbered set (prefix, from, to, up to 500) or one badge; **Edit** number, type and note (the number is locked while the badge is in use); **Retire** and **Restore**; **Delete** with a reason, recorded in `config/badges.removed` for the access-device log.
- **Status** per badge comes from the visits: in use (with holder and time), available, not returned (a clock-out recorded it missing), retired.
- Each row shows its **NFC tag** (last 6 characters) with **Link tag**, **Relink** and **Unlink**, and its **link for the QR code** with **Copy**. **Download all links** saves a CSV (Badge, Type, Link); **Print all QR labels** makes a printable sheet.

QR codes on badges

Each badge's QR code holds exactly `<web app URL>?badge=<number>`. Generate them in byte mode so letter case is kept exactly: Apps Script deployment ids are case-sensitive, and some online QR generators uppercase or trim long links, which produces "Sorry, unable to open the file at this time." Verify every code by decoding it back before printing. Print at 100%, about 1.4 inches square with error correction M.

The link must never change. It's the web app deployment's address. Update code only through Deploy, Manage deployments, Edit (pencil), New version. **New deployment** creates a new address and orphans every printed code; archiving the deployment kills it.

NFC tap-to-sign-out

Each badge card carries an NFC sticker. A generic USB NFC reader acts as a keyboard: it types the tag's number and presses Enter. The Visitor Log listens to every keydown (capture phase) and recognizes a tap by timing:

- **A tap** is 4 or more characters typed with an average gap under 45 ms, with Enter coming within 150 ms of the last character. People type far slower, so normal typing is never mistaken for a tap.
- On a tap, the Enter is cancelled, and if a text box had focus, the typed characters are removed from it.
- **Normalizing the tag number:** uppercase, keep only A to Z and 0 to 9. Some readers send the number twice in one read (`7475bb537475bb53`); when the string is 8 or more characters and its two halves are equal, keep one half (`7475BB53`). The same normalization runs when linking and when tapping, so both always agree.
- **The same tag within 3 seconds** is ignored as a double read.

Linking tags (Editors): Link NFC tags walks through every badge without a tag in order ("Tap badge V1's card on the reader"), with **Skip**, **Stop** and a box to type a tag number by hand. A tag already on another badge moves to the new one, and the message says so. Tags are stored as `nfc` on the badge in `config/badges`.

On a tap:

Situation	What happens	Feedback
Tag linked, badge checked out to a visitor	Checkout written with the badge returned; event "Signed out by tapping badge V1 at the reader; badge returned."	Large card with photo: "Pat Lee is signed out", two-tone beep, **Undo**, closes after 5 s.
Tag linked, badge not checked out	Nothing written.	"Badge V1 isn't checked out", low tone.
Tag not linked	Nothing written.	"This tag isn't linked to a badge", low tone.
Viewer-level screen	Nothing written.	"Can't sign out here".

The reader types into whichever window has keyboard focus. On a normal screen, someone must have clicked into the Visitor Log; the kiosk solves this.

The sign-out kiosk

Started from **Sign-out kiosk** in the top bar (Contributors and up). A full-screen layer over the app:

- **Header:** "Rehtek visitor sign-out", the time (updated every 15 s), and an **Exit** button.
- **Banner:** "Tap your badge on the reader to sign out. Or find your name below and press Clock out. Please leave your badge at the front desk."
- **A card per visitor inside,** A to Z: photo or initials, name, company, badge number or "Digital pass", time in, and a large red **Clock out**. Nothing else is shown, because anyone in the lobby can read it (no citizenship, no host).
- **Clock out** on a physical badge asks "Leaving badge V2 at the front desk?" with **Yes, sign me out** (returned) or **I don't have it** (not returned, incident logged); a digital pass just confirms. The confirmation resets after 15 s. Every sign-out shows the large card with **Undo**.

How it stays reliable:

- **Full screen** through the Fullscreen API. Browsers always let Esc leave full screen, so the kiosk stays up and shows **Go back to full screen**. Where the page isn't allowed to go full screen, it says to press F11 (Windows) or Control-Command-F (Mac).
- **Keyboard focus:** an invisible text input is focused at start and refocused every 800 ms (unless a dialog is open), so reader taps always land in the app.
- **Locked: Exit** asks for the staff PIN, checked against the stored hash; 5 wrong tries pause for a minute. With no PIN set, it asks for confirmation instead.
- **Survives reloads:** a local-storage flag brings the kiosk back after a refresh or the nightly reload.
- **Screen stays on:** it requests a screen wake lock where the browser allows it.

9. Compliance features

Citizenship gate and Empowered Official approval

- Every visitor states citizenship status at sign-in (or the desk records it at pre-registration). The front desk confirms it against the photo ID at check-in (`citizenshipConfirmed`).
- **Foreign persons are never admitted automatically.** Their visit waits with reason `approval` until the Empowered Official records a decision in `approvals/{visitId}`; on the next poll the visitor is admitted, with their held badge if they took one, and their phone flips to "checked in".
- **Decisions:** approved; approved for non-controlled areas only ("limited", which strips controlled areas from what the badge board shows and blocks controlled doors); denied (stays held, and the desk dismisses it). ITAR and EAR release fields and a license number are recorded with each decision.
- Approval is per visit. A returning foreign national needs a decision for each new visit. Countries under 22 CFR 126.1 should be flagged by the Empowered Official in the approval note.

Consolidated Screening List

- The Empowered Official downloads the U.S. government Consolidated Screening List CSV (from trade.gov's downloadable consolidated list) and uploads it in Settings, at least weekly. The app parses it, stores it as a JSON file in its file store, and points `config/screening` at it. Settings warns when it's older than 7 days.

- **Search:** the visitor's name and company are split into tokens and compared with each list entry's names and alternate names using `near` (one edit allowed for tokens of 5 or more letters). A hit needs most name tokens to match. Results show the matched entry, its source list and the matching words.
- At admission, a possible match holds the visitor with reason `screening`; the Empowered Official reviews it and clears it (recorded as `screening: "possible"` with a decision) or denies it. Pre-registrations are screened in the approval dialog. The check-in dialog shows the automatic result, or asks for a manual confirmation with a link to trade.gov's online search when the list isn't loaded.

Escorts (CMMC 3.10.3, Technology Control Plan)

- A named escort is recorded at check-in. Foreign nationals require a U.S.-person escort at all times; the check-in dialog and the escort change dialog both require ticking that confirmation (`tcpEscort`).
- **Escort handover:** a staff member scans the visitor's badge and taps **Take over as escort**; the change is an `escort` event.
- **Host alert:** the host gets an in-app arrival banner within 30 minutes of the check-in.

Areas and door scans

- Areas are a fixed list with a `controlled` flag (for example lobby and offices, shop floor, inspection, shipping). Each door gets a printed QR sign `RVL:A:<area>`.
- Scanning a door, then a badge, logs an `area` event ("Entered shop floor") and updates the last-seen area shown on the badge board.
- **Blocking:** a foreign national approved for non-controlled areas only, scanned at a controlled door, gets a red refusal and an incident event.

Audit trail (CMMC 3.10.4)

- Visits are never deleted. Every change appends an event with time and user. Corrections keep the old value in a `correction` event.
- **Monthly review:** Records shows the month's counts (visits, foreign-person visits, visits without approval, missing escorts, badges not returned, overdue clock-outs); an Editor marks it reviewed in `reviews/{YYYY-MM}`.
- **CSV export** of any range, including areas entered, escort changes, screening and release decisions.
- **Retention:** keep visit records and the sign-in spreadsheet for 5 years.

- **Overdue:** a visitor still checked in after their leave time, or from a previous day, is flagged on Inside now.

Access devices (CMMC 3.10.5)

The badge register, type enforcement at check-in, return confirmation at every clock-out (desk, kiosk or NFC tap), not-returned incidents, and retirement and deletion with reasons. Digital passes expire at checkout.

Visitor acknowledgment

The five rules (section 5) are the agreement. Checking the box is the electronic signature: the visitor's name becomes `ackName`, with `ackAt`, `ackSource` ("badge page" or the sign-in page) and `ackVersion`. Front desk re-entries reconfirm the rules verbally and record "Re-confirmed at the front desk".

10. Security, privacy and abuse limits

What the public page will and won't reveal

- **Never a list of visitors.** Lookups need a first and last name plus a company; only matching people come back, and only their own details.
- **Status checks never name anyone.** A free badge shows nothing about past holders.
- **A badge in use shows photo, name and badge number to anyone who scans it.** This is Rehtek's deliberate choice; it means anyone holding a badge link, even outside the building, can see who is wearing that badge. Everything else (company, escort, host, areas, citizenship kind, time in) is behind the staff PIN.
- **Tokens instead of ids:** `findPeople` hands the phone a random token per match, stored server-side for 15 minutes; every later call uses the token, so a phone can never fetch an arbitrary directory record by guessing ids.

Rate limits (Apps Script CacheService, shared across all phones, 10-minute windows)

Action	Limit	After the limit
Wrong staff PIN	10	All PIN checks refused for the rest of the window, plus a 0.7 s delay on every wrong try.
Name searches and new sign-ins	60	"The badge page is busy. Please wait a minute, or see the front desk."
Duplicate requests	1 per badge and action per 10 minutes	The existing request id is returned instead of a new row.

PINs

- **Badge page staff PIN:** a script property (`PIN`) in the Apps Script project, compared on the server only. Use 6 to 8 digits; a 4-digit PIN can be guessed in days even with the lockout.
- **In-app staff PIN** (badge lookup screen and kiosk exit): stored as `{ salt, hash }` with hash = SHA-256 of `salt:PIN`, salt 16 random bytes. It's a convenience lock on shared screens, not strong security: anyone with app access can read the hash.

Input handling

- Every visitor-typed value is trimmed, whitespace-collapsed and cut to 120 characters on the server, and leading `= + - @` are removed before it reaches a sheet, so it can never run as a formula. CSV exports prefix such cells with an apostrophe.
- All visitor text shown in the Visitor Log is HTML-escaped. The public page builds its screens with DOM methods and `textContent`, never by injecting HTML.
- Photos are accepted only as JPEG, PNG or WebP data URLs, checked by pattern before display or saving; photo data on the badge board is shown only if it matches the image data-URL pattern.
- Badge numbers in links are cleaned to letters, digits, `_ . ~ -`, 20 characters at most.

Access and data protection

- **The Visitor Log is never shared by public link.** Contributors work it; Viewers see only badge lookups; Editors hold approvals, settings and the register (enforced by the database rules in section 3).
- **The spreadsheet and photo folder** belong to the shop's Google account and are shared only with front desk staff. The Apps Script runs as that account, which is why it can write rows and read photos without the visitor having any access.
- **Personal data** (names, photos, citizenship) is used only to manage visits and meet export rules, as the intro text promises. Visitors are told not to enter ID numbers. Photos and citizenship are personal data in scope for the CMMC assessment boundary.
- **Sign-in leases and badge locks** prevent two front desk screens from admitting the same person twice or issuing one badge to two people.

11. Deployment, setup and pitfalls

Setting it up from scratch

1. **Spreadsheet:** create a Google spreadsheet owned by the shop's account. Add a tab named **Form Responses 1** with the 11 headers from section 4 in row 1. Share it only with front desk staff.
2. **Photo folder:** create a Drive folder for visitor photos; note its id.
3. **Apps Script:** in the spreadsheet, Extensions, Apps Script. Paste the sign-in page code (section 5). Project Settings, Script properties: `PIN` (6 to 8 digits) and `PHOTO_FOLDER_ID`. Run `setup` once and allow access to Sheets and Drive.
4. **Deploy:** Deploy, New deployment, type Web app, Execute as **Me**, Who has access **Anyone**. Copy the URL ending in `/exec`. This is the **only** time you use "New deployment".
5. **Visitor Log:** a single-page web app with a document database (rules from section 3), a file store, downloads, the signed-in user's identity, and the Google Sheets (read, append, update values) and Google Drive (download file content) connectors. Open it as an Editor.
6. **Settings:** paste the spreadsheet link (Visitor sign-in), the `/exec` URL (Public badge links) and set the in-app staff PIN.

7. **Badges:** add V1 to V25 (type `any`), print the QR labels from the exact links, and verify a few by scanning. Link each card's NFC tag with **Link NFC tags**.
8. **Lobby:** print the lobby QR poster (Settings, Visitor sign-in, Show the lobby QR code), the door signs, and set up the lobby screen with the NFC reader running the **Sign-out kiosk**.
9. **Screening:** upload the Consolidated Screening List CSV, then weekly.
10. **Test end to end:** a new visitor from a badge QR, a returning visitor, a foreign national (approval), a badge-in-use scan with See more, an NFC tap, a kiosk Clock out.

Updating the sign-in page later

Paste the new code, Save, then Deploy, **Manage deployments**, select the existing deployment, pencil (Edit), Version **New version**, Deploy. The URL and every printed QR code stay the same. Run `setup` again only when tabs or headers change.

Pitfalls found while building it (each one broke the real system once)

Pitfall	Symptom	Fix
Backslashes inside the HTML template string. Apps Script serves `PAGE` as a JavaScript template literal, which turns `\/` into `/`, `\s` into `s` and `\"` into `"`.	The page shows only its header: the script has a syntax error and nothing runs.	Write every backslash in page code doubled (`\\/`), or avoid escapes (use typographic quotes in text). Test by evaluating `PAGE` exactly as Apps Script does and syntax-checking the resulting script, not the source text.
New deployment instead of a new version.	Printed QR codes open old code, or a second link appears.	Always Manage deployments, Edit, New version. Archive any stray deployment.
QR generators that change the link.	"Sorry, unable to open the file at this time" when scanned, while the pasted link works.	Generate codes in byte mode from the exact link; decode every code back before printing.

Pitfall	Symptom	Fix
Several Google accounts in one phone browser.	The same "unable to open the file" error.	Use Anyone access (not "Anyone with a Google account"); test in a private window.
Screens left open for days run old code.	A new feature (for example the People tab) never appears.	Nightly self-reload at 3 AM; kiosk mode restores itself after reload.
People tab empty or behind.	Returning visitors come back "not on file".	The page also searches sign-in history in Form Responses 1.
Auto-assigning a physical badge to form sign-ins.	The system gives badge V4 while the visitor holds V2.	Form sign-ins start on a digital pass; the scanned or held badge replaces it.
Two-cycle admission for new visitors.	30 to 60 seconds before "checked in".	Read held badges before processing sign-ins, so admission gives the held badge in one step; poll every 10 s.
Large photos.	Slow uploads; a photo too big for the card pushes text off screen.	Resize on the phone to 900 px JPEG; give grid children `min-width: 0` and the photo `overflow: hidden`.
Absolutely positioned initials inside a circle with no positioned parent.	An invisible layer covers the whole card, and its OK button can't be pressed.	`position: relative` on the circle.
Browser dialogs in the Apps Script sandbox.	`confirm()` may never appear.	Use a tap-again button instead.
Keyboard-style NFC readers.	Taps type into the chat box or another window; one reader sends the number twice.	Kiosk focus trap; normalize doubled numbers; detect taps by timing.
Unordered database limits.	Past 1,000 records, recent approvals or sign-ins disappear from view.	Always bound queries by date and order them; read long ranges month by month.

Pitfall	Symptom	Fix
Formula injection.	A visitor named "=HYPERLINK(…)" becomes a live formula in the sheet or a CSV.	Strip leading `= + - @` before writing; prefix in CSV exports.
Inconsistent time formats.	Old form rows and new page rows sort or parse differently.	Write `M/D/YYYY H:MM:SS` in the shop's time zone everywhere.

Testing approach that works

Simulate both sides against one shared fake spreadsheet: run the real Apps Script server functions with stand-ins for SpreadsheetApp, DriveApp, CacheService, LockService and Utilities, serve the page exactly as `doGet` builds it, and run the real Visitor Log against a fake database and Sheets connector backed by the same storage. Then walk the scenarios: a returning visitor by name and company, a new visitor through the in-app sign-in, a visitor already signed in taking a badge, switching badges, a foreign national approved while waiting, a wrong-type badge, a badge-in-use scan with See more, an NFC tap with a double read, and a kiosk Clock out with Undo.